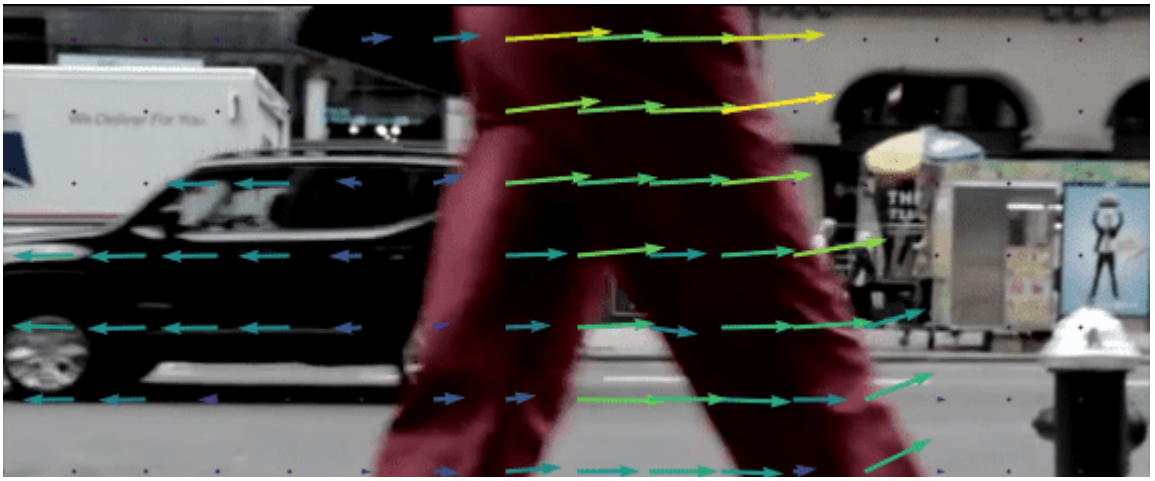
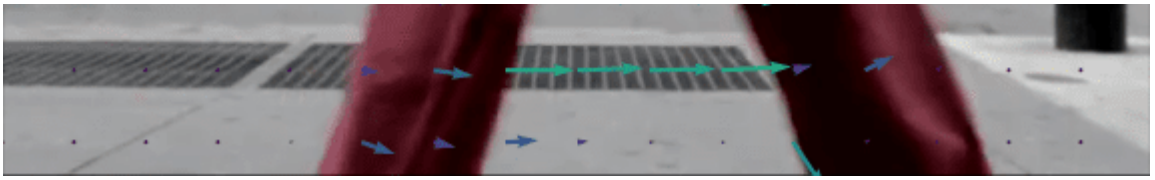
Optical Flow in OpenCV (C++/Python)

In this post, we will learn about the various algorithms for calculating Optical Flow in a video or sequence of frames. We will discuss the relevant theory and implementation in OpenCV of sparse and dense optical flow algorithms. We share code in C++ and Python. Specifically, you will learn the following:

What is Optical Flow?

Optical flow is a task of per-pixel motion estimation between two consecutive frames in one video. Basically, the Optical Flow task implies the calculation of the shift vector for pixel as an object displacement difference between two neighboring images. The main idea of Optical Flow is to estimate the object's displacement vector caused by it's motion or camera movements.

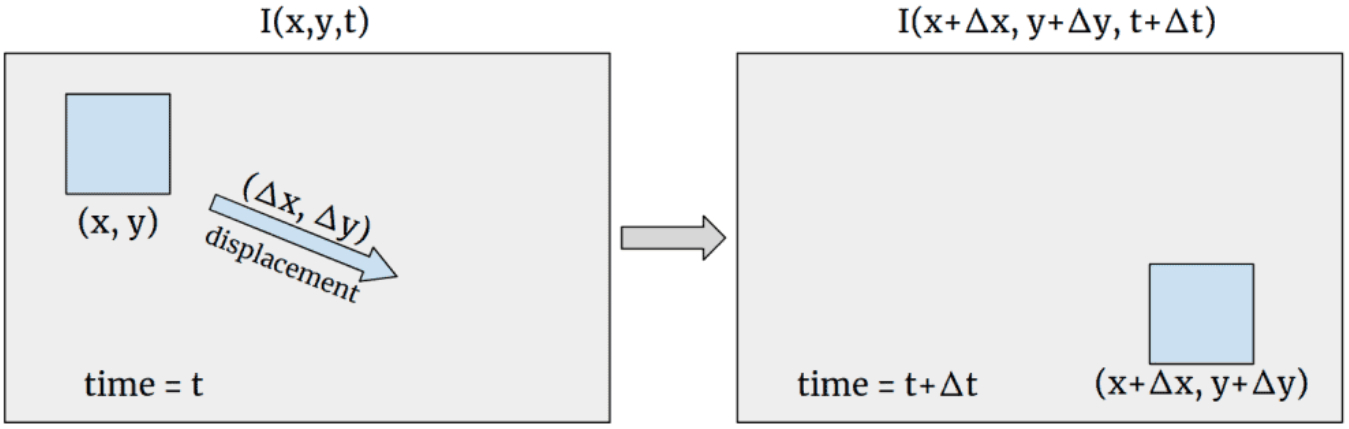




Theoretical basics

Let's assume that we have a gray-scale image – the matrix with pixel intensity. We define the function $I(x, y, t)$, where x, y – pixel coordinates and t – frame number. The $I(x, y, t)$ function defines the exact pixel intensity at the frame t.

To start with, we assume that the object displacement doesn't change the pixels intensity that belongs to the exact object, it means that $I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$. In our case $\Delta t = 1$. The major concern is to find the motion vector $(\Delta x, \Delta y)$. Let's take a look at graphical representation:



Using Taylor series expansion we can rewrite $I(x, y, t) - I_(x + \Delta x, y + \Delta y, t + \Delta t) = 0$ as $I'_x u + I'_y v = -I'_t$, where $u = \frac{dx}{dt}, v = \frac{dy}{dt}$, and I'_x, I'_y are image gradients. It is important that here we assume that parts of higher-order Taylor series are negligible, so this is a function approximation using only first-order Taylor's expansion. The pixel motion difference between two frames I_1 and I_2 can be written as $I_1 - I_2 \approx I'_x u + I'_y v + I'_t$. Now, we have two variables u, v and only one equation, so we can't solve the equation right now, but we can use some tricks which will be disclosed in the following algorithms.

Optical Flow applications

Optical Flow can be used in many areas where the object's motion information is crucial. Optical Flow is commonly found in video editors for compression, stabilization, slow-motion, etc. Also, Optical Flow finds its application in Action Recognition tasks and real-time tracking systems.

Sparse and Dense Optical Flow

There are two types of Optical Flow, and the first one is called Sparse Optical Flow. It computes the motion vector for the specific set of objects (for example – detected corners on image). Hence, it requires some preprocessing to extract features from the image, which will be the basement for the Optical Flow calculation. OpenCV provides some algorithm implementations to solve the Sparse Optical Flow task:

Pyramid Lucas-Kanade

Sparse RLOF

Using only a sparse feature set means that we will not have the motion information about pixels that are not contained in it. This restriction can be lifted using Dense Optical Flow algorithms which are supposed to calculate a motion vector for every pixel in the image. Some Dense Optical Flow algorithms are already implemented in OpenCV:

Dense Pyramid Lucas-Kanade

Farneback

PCAFlow

SimpleFlow

RLOF

DeepFlow

DualTVL1

In this post we will take a look at the theoretical aspects of some of these algorithms and their usage with OpenCV.

Sparse Optical Flow

Lucas-Kanade algorithm

The Lucas-Kanade method is commonly used to calculate the Optical Flow for a sparse feature set. The main idea of this method based on a local motion constancy assumption, where nearby pixels have the same displacement direction. This assumption helps to get the approximated solution for the equation with two variables.

Theory